Aula 4 — Histórico e análise de leituras

Componente curricular

Linguagens de Programação e Banco de Dados

Curso

Curso Superior de Tecnologia em Automação Industrial

Instituição

Instituto Federal do Paraná — Campus Assis Chateaubriand

Unidade curricular

Unidade 4 — Estruturas de dados aplicadas ao histórico de sensores

Duração

4 horas-aula

1. Situação-problema

Nas aulas anteriores, o programa foi capaz de:

• Receber dados de um motor;

• Classificar temperatura, pressão e corrente;

• Simular vários ciclos de coleta;

• Identificar situações normais, de atenção, alarme e emergência;

• Interromper a coleta em caso de emergência;

• Ignorar uma leitura com falha de comunicação.

Entretanto, depois que cada leitura é exibida na tela, ela deixa de estar disponível para análise.

O supervisor da planta deseja obter informações como:

• Maior temperatura registrada;

• Menor temperatura registrada;

• Temperatura média;

• Quantidade de leituras acima do limite;

• Últimas leituras realizadas;

• Estado mais frequente do equipamento;

• Ciclo em que ocorreu o maior valor;

• Tendência de aumento ou redução da temperatura;

• Quantidade de leituras armazenadas.

Para responder a essas perguntas, o programa deverá armazenar as leituras durante a execução.

O desafio desta aula é transformar o simulador da Aula 3 em um sistema capaz de **armazenar, consultar e analisar o histórico das coletas**.

2. Objetivos da aula

Ao final da aula, o estudante deverá ser capaz de:

• Criar e utilizar listas em Python;

• Armazenar valores coletados durante a execução;

• Adicionar elementos utilizando `append()`;

• Consultar elementos por meio de índices;

• Utilizar índices positivos e negativos;

• Utilizar fatiamento de listas;

• Percorrer listas com `for`;

• Calcular quantidade, menor valor, maior valor e média;

• Contar ocorrências;

• Identificar leituras acima de um limite;

• Relacionar os valores armazenados aos ciclos correspondentes;

• Analisar a tendência de uma variável;

• Produzir um relatório do histórico de um equipamento.

3. Produto final da aula

Ao final do encontro, cada estudante ou dupla deverá possuir um programa que:

1. Simule vários ciclos de coleta;

2. Armazene as leituras dos sensores;

3. Armazene os números dos ciclos válidos;

4. Armazene os estados gerais;

5. Exiba o histórico coletado;

6. Calcule estatísticas dos sensores;

7. Conte leituras acima dos limites;

8. Identifique os ciclos críticos;

9. Analise a tendência da temperatura;

10. Apresente um resumo final da coleta.

4. Conhecimentos utilizados

Esta aula utilizará conceitos trabalhados anteriormente:

• Variáveis;

• Tipos de dados;

• Operadores;

• Estruturas condicionais;

• Estruturas de repetição;

• Simulação com `random`;

• Classificação de estados;

• Contadores;

• Comandos `break` e `continue`.

A principal novidade será o uso de **listas para armazenar vários valores relacionados**.

5. Organização da aula

Etapa	Atividade	Tempo sugerido
1	Retomada da Aula 3 e apresentação do problema	20 min
2	Desafio 1 — criando listas de leituras	25 min
3	Índices e acesso aos elementos	20 min
4	Desafio 2 — armazenando os ciclos simulados	35 min
5	Desafio 3 — exibindo o histórico	25 min
6	Desafio 4 — calculando estatísticas	35 min
7	Desafio 5 — contando leituras críticas	30 min
8	Intervalo ou pausa técnica	10 min
9	Desafio 6 — analisando as últimas leituras	25 min
10	Desafio 7 — identificando ciclos críticos	30 min
11	Desafio 8 — analisando tendências	25 min
12	Integração, testes e avaliação	40 min
Total		320 min

Os tempos podem ser ajustados conforme a duração adotada para cada hora-aula e o ritmo da turma.

6. Retomada da Aula 3

Abra o programa `simulador_coleta.py` desenvolvido na aula anterior.

Na Aula 3, o sistema realizava leituras semelhantes a:

```
CICLO 1
Temperatura: 72 °C
Pressão: 7.2 bar
Corrente: 13.5 A
Estado geral: ATENÇÃO
```

```
CICLO 2
Temperatura: 68 °C
Pressão: 6.5 bar
Corrente: 11.8 A
Estado geral: NORMAL
```

Porém, ao final da execução, o programa não consegue responder:

```
Qual foi a maior temperatura?
```

Também não consegue responder:

```
Em qual ciclo ocorreu a maior temperatura?
```

Ou ainda:

```
Quantas leituras ultrapassaram o limite?
```

Para responder a essas perguntas, os valores deverão ser armazenados durante os ciclos de coleta.

Atividade inicial

Analise o seguinte trecho:

```
for ciclo in range(5):
 temperatura = random.randint(60, 95)
 print(temperatura)
```

O valor da temperatura é exibido, mas não é armazenado.

Modifique o programa para guardar as temperaturas em uma lista.

7. Desafio 1 — Criando listas de leituras

Problema

Crie uma lista para armazenar as temperaturas coletadas.

```
historico_temperatura = []
```

Neste momento, a lista está vazia.

Para adicionar um valor:

```
historico_temperatura.append(72)
```

Adicione outros valores:

```
historico_temperatura.append(68)
historico_temperatura.append(75)
historico_temperatura.append(81)
```

Exiba a lista:

```
print(historico_temperatura)
```

Resultado esperado:

```
[72, 68, 75, 81]
```

Armazenando outras variáveis

```
historico_pressao = []
historico_corrente = []
historico_estados = []
```

Adicionando dados:

```
historico_pressao.append(6.5)
historico_corrente.append(12.0)
historico_estados.append("NORMAL")
```

Programa de teste

```
historico_temperatura = []
historico_pressao = []
historico_corrente = []
historico_estados = []
```

```
historico_temperatura.append(72)
historico_pressao.append(6.5)
historico_corrente.append(12.0)
historico_estados.append("NORMAL")

print(historico_temperatura)
print(historico_pressao)
print(historico_corrente)
print(historico_estados)
```

Atividade

Crie listas para armazenar:

• Temperatura;

• Pressão;

• Corrente;

• Estado geral;

• Número do ciclo.

```
historico_ciclos = []
historico_temperatura = []
historico_pressao = []
historico_corrente = []
historico_estados = []
```

8. Índices das listas

Os elementos de uma lista são identificados por índices.

Em Python, o primeiro índice é `0`.

```
historico_temperatura = [72, 68, 75, 81]
```

Índice	Valor
0	72
1	68

2	75
3	81

Para acessar os valores:

```
print(historico_temperatura[0])
print(historico_temperatura[1])
print(historico_temperatura[3])
```

Para acessar o último valor:

```
print(historico_temperatura[-1])
```

Para acessar o penúltimo valor:

```
print(historico_temperatura[-2])
```

Exemplo completo

```
leituras = [63, 67, 72, 78, 81, 76]

print(f"Primeiro valor: {leituras[0]}")
print(f"Terceiro valor: {leituras[2]}")
print(f"Último valor: {leituras[-1]}")
print(f"Penúltimo valor: {leituras[-2]}")
```

Atenção

O índice precisa existir na lista.

O código abaixo gera erro:

```
leituras = [63, 67, 72]

print(leituras[5])
```

A lista possui apenas os índices `0`, `1` e `2`.

9. Desafio 2 — Armazenando os ciclos simulados

Problema

O programa deverá armazenar todas as leituras de temperatura, pressão, corrente e estado geral.

Antes do laço, crie as listas:

```
historico_ciclos = []
historico_temperatura = []
historico_pressao = []
historico_corrente = []
historico_estados = []
```

Dentro do laço, depois de gerar e classificar uma leitura, adicione os valores:

```
historico_ciclos.append(numero_ciclo)
historico_temperatura.append(temperatura)
historico_pressao.append(pressao)
historico_corrente.append(corrente)
historico_estados.append(estado_geral)
```

Exemplo

```
import random

historico_ciclos = []
historico_temperatura = []
historico_pressao = []
historico_corrente = []
historico_estados = []

quantidade_ciclos = 5

for ciclo in range(quantidade_ciclos):
 numero_ciclo = ciclo + 1

 temperatura = random.randint(60, 95)
 pressao = random.uniform(5.0, 10.0)
 corrente = random.uniform(10.0, 20.0)
```

```
 estado_geral = "EM ANÁLISE"

 historico_ciclos.append(numero_ciclo)
 historico_temperatura.append(temperatura)
 historico_pressao.append(pressao)
 historico_corrente.append(corrente)
 historico_estados.append(estado_geral)

print(historico_ciclos)
print(historico_temperatura)
print(historico_pressao)
print(historico_corrente)
print(historico_estados)
```

Armazenamento dentro do programa da Aula 3

Depois de determinar o `estado_geral`, acrescente:

```
historico_ciclos.append(numero_ciclo)
historico_temperatura.append(temperatura)
historico_pressao.append(pressao)
historico_corrente.append(corrente)
historico_estados.append(estado_geral)
```

Leitura inválida

Quando uma leitura for ignorada, nenhuma lista deverá receber dados naquele ciclo.

```
if numero_ciclo == 5:
 print("Falha de comunicação. Leitura ignorada.")
 leituras_ignoradas += 1
 continue
```

Dessa forma, o ciclo 5 não será incluído no histórico de leituras válidas.

10. Desafio 3 — Exibindo o histórico

Exibição simples

```
print(historico_temperatura)
```

Embora funcione, esse formato não é adequado para um relatório industrial.

Exibição com `for`

```
for temperatura in historico_temperatura:
 print(f"Temperatura registrada: {temperatura:.2f} °C")
```

Exibindo o índice e o valor

```
for indice in range(len(historico_temperatura)):
 temperatura = historico_temperatura[indice]

 print(
 f"Leitura {indice + 1}: "
 f"{temperatura:.2f} °C"
)
```

Exibindo as listas relacionadas

```
for indice in range(len(historico_temperatura)):
 print()
 print(f"Ciclo: {historico_ciclos[indice]}")
 print(
 f"Temperatura: "
 f"{historico_temperatura[indice]:.2f} °C"
)
 print(
 f"Pressão: "
 f"{historico_pressao[indice]:.2f} bar"
)
 print(
 f"Corrente: "
 f"{historico_corrente[indice]:.2f} A"
)
 print(f"Estado: {historico_estados[indice]}")
```

Atenção

As listas relacionadas precisam permanecer alinhadas:

```
Índice 0 ' leitura do ciclo armazenado em historico_ciclos[0]
Índice 1 ' leitura do ciclo armazenado em historico_ciclos[1]
Índice 2 ' leitura do ciclo armazenado em historico_ciclos[2]
```

A lista `historico_ciclos` é importante porque podem existir ciclos ignorados.

11. Desafio 4 — Calculando estatísticas

Problema

O supervisor deseja conhecer os valores mínimo, máximo e médio de cada sensor.

Temperatura

```
menor_temperatura = min(historico_temperatura)
maior_temperatura = max(historico_temperatura)
media_temperatura = (
 sum(historico_temperatura)
/ len(historico_temperatura)
)
```

Exibição:

```
print(f"Menor temperatura: {menor_temperatura:.2f} °C")
print(f"Maior temperatura: {maior_temperatura:.2f} °C")
print(f"Média da temperatura: {media_temperatura:.2f} °C")
```

Pressão

```
menor_pressao = min(historico_pressao)
maior_pressao = max(historico_pressao)
media_pressao = (
 sum(historico_pressao)
```

```
/ len(historico_pressao)
)
```

Corrente

```
menor_corrente = min(historico_corrente)
maior_corrente = max(historico_corrente)
media_corrente = (
 sum(historico_corrente)
/ len(historico_corrente)
)
```

Relatório estatístico

```
print()
print("========================================")
print(" ESTATÍSTICAS DO HISTÓRICO")
print("========================================")
print(
 f"Leituras armazenadas: "
 f"{len(historico_temperatura)}"
)

print()
print("TEMPERATURA")
print(f"Mínima: {menor_temperatura:.2f} °C")
print(f"Máxima: {maior_temperatura:.2f} °C")
print(f"Média: {media_temperatura:.2f} °C")

print()
print("PRESSÃO")
print(f"Mínima: {menor_pressao:.2f} bar")
print(f"Máxima: {maior_pressao:.2f} bar")
print(f"Média: {media_pressao:.2f} bar")

print()
print("CORRENTE")
print(f"Mínima: {menor_corrente:.2f} A")
print(f"Máxima: {maior_corrente:.2f} A")
print(f"Média: {media_corrente:.2f} A")
```

Verificação de lista vazia

As funções `min()`, `max()` e `sum()` não devem ser utilizadas com uma lista vazia.

```
if len(historico_temperatura) > 0:
 menor_temperatura = min(historico_temperatura)
 maior_temperatura = max(historico_temperatura)
 media_temperatura = (
 sum(historico_temperatura)
/ len(historico_temperatura)
)

 print(f"Mínima: {menor_temperatura:.2f} °C")
 print(f"Máxima: {maior_temperatura:.2f} °C")
 print(f"Média: {media_temperatura:.2f} °C")
else:
 print("Nenhuma leitura de temperatura foi armazenada.")
```

12. Desafio 5 — Contando leituras acima dos limites

Problema

O supervisor deseja saber quantas leituras ultrapassaram os limites de segurança.

Considere:

```
limite_temperatura = 80
limite_pressao = 8
limite_corrente = 15
```

Temperatura

```
leituras_temperatura_alta = 0

for temperatura in historico_temperatura:
 if temperatura > limite_temperatura:
 leituras_temperatura_alta += 1
```

Pressão

```
leituras_pressao_alta = 0

for pressao in historico_pressao:
 if pressao > limite_pressao:
 leituras_pressao_alta += 1
```

Corrente

```
leituras_corrente_alta = 0

for corrente in historico_corrente:
 if corrente > limite_corrente:
 leituras_corrente_alta += 1
```

Exibição

```
print()
print("LEITURAS ACIMA DOS LIMITES")
print("----------------------------------------")
print(
 f"Temperatura: "
 f"{leituras_temperatura_alta}"
)
print(f"Pressão: {leituras_pressao_alta}")
print(f"Corrente: {leituras_corrente_alta}")
```

Percentual de leituras acima do limite

```
total_leituras = len(historico_temperatura)

if total_leituras > 0:
 percentual_temperatura = (
 leituras_temperatura_alta
/ total_leituras
* 100
)
```

```
 print(
 f"Percentual de temperaturas acima "
 f"do limite: "
 f"{percentual_temperatura:.2f}%"
)
```

Observação

Uma leitura exatamente igual ao limite não está acima dele.

Por exemplo:

```
temperatura = 80
```

Não satisfaz:

```
temperatura > 80
```

13. Desafio 6 — Analisando as últimas leituras

Problema

O operador deseja consultar apenas as três leituras mais recentes.

Utilize o fatiamento:

```
ultimas_temperaturas = historico_temperatura[-3:]
```

Exibição:

```
print("ÚLTIMAS TEMPERATURAS")

for temperatura in ultimas_temperaturas:
 print(f"{temperatura:.2f} °C")
```

Últimos ciclos completos

```
inicio_ultimas = max(
0,
 len(historico_ciclos) - 3
)

for indice in range(
 inicio_ultimas,
 len(historico_ciclos)
):
 print()
 print(f"Ciclo: {historico_ciclos[indice]}")
 print(
 f"Temperatura: "
 f"{historico_temperatura[indice]:.2f} °C"
)
 print(
 f"Pressão: "
 f"{historico_pressao[indice]:.2f} bar"
)
 print(
 f"Corrente: "
 f"{historico_corrente[indice]:.2f} A"
)
 print(f"Estado: {historico_estados[indice]}")
```

Outros exemplos de fatiamento

Primeiras três leituras

```
primeiras_temperaturas = historico_temperatura[:3]
```

Leituras do índice 2 até o índice 5

```
parte_historico = historico_temperatura[2:6]
```

Leituras alternadas

```
leituras_alternadas = historico_temperatura[::2]
```

Histórico em ordem inversa

```
historico_invertido = historico_temperatura[::-1]
```

Teste

```
historico = [60, 62, 65, 70, 75, 78, 82, 85]

print("Primeiras três:", historico[:3])
print("Últimas três:", historico[-3:])
print("Parte do histórico:", historico[2:6])
print("Leituras alternadas:", historico[::2])
print("Histórico invertido:", historico[::-1])
```

14. Desafio 7 — Identificando ciclos críticos

Problema

O supervisor deseja saber em qual ciclo ocorreu a maior temperatura.

A função `max()` informa o maior valor:

```
maior_temperatura = max(historico_temperatura)
```

Para encontrar o índice:

```
indice_maior_temperatura = (
 historico_temperatura.index(
 maior_temperatura
)
)
```

Para recuperar o número original do ciclo:

```
ciclo_maior_temperatura = (
 historico_ciclos[indice_maior_temperatura]
)
```

Exibição

```
print(
 f"Maior temperatura: "
 f"{maior_temperatura:.2f} °C"
)

print(
 f"Ciclo da maior temperatura: "
 f"{ciclo_maior_temperatura}"
)
```

Maior pressão

```
maior_pressao = max(historico_pressao)

indice_maior_pressao = (
 historico_pressao.index(
 maior_pressao
)
)

ciclo_maior_pressao = (
 historico_ciclos[indice_maior_pressao]
)
```

Maior corrente

```
maior_corrente = max(historico_corrente)

indice_maior_corrente = (
 historico_corrente.index(
 maior_corrente
)
)
```

```
ciclo_maior_corrente = (
 historico_ciclos[indice_maior_corrente]
)
```

Verificação de segurança

```
if len(historico_temperatura) > 0:
 maior_temperatura = max(historico_temperatura)

 indice_maior_temperatura = (
 historico_temperatura.index(
 maior_temperatura
)
)

 ciclo_maior_temperatura = (
 historico_ciclos[
 indice_maior_temperatura
]
)

 print(
 f"Maior temperatura: "
 f"{maior_temperatura:.2f} °C"
)

 print(
 f"Ciclo: "
 f"{ciclo_maior_temperatura}"
)
```

15. Desafio 8 — Analisando a tendência da temperatura

Problema

O supervisor deseja saber se a temperatura aumentou ou diminuiu entre a primeira e a última leitura.

Comparação entre a primeira e a última leitura

```
if len(historico_temperatura) >= 2:
 primeira_temperatura = (
 historico_temperatura[0]
)

 ultima_temperatura = (
 historico_temperatura[-1]
)

 variacao_temperatura = (
 ultima_temperatura
- primeira_temperatura
)

 if variacao_temperatura > 0:
 tendencia = "AUMENTO"
 elif variacao_temperatura < 0:
 tendencia = "REDUÇÃO"
 else:
 tendencia = "ESTÁVEL"

 print()
 print("TENDÊNCIA DA TEMPERATURA")
 print("----------------------------------------")
 print(
 f"Primeira leitura: "
 f"{primeira_temperatura:.2f} °C"
)
 print(
 f"Última leitura: "
 f"{ultima_temperatura:.2f} °C"
)
 print(
 f"Variação: "
 f"{variacao_temperatura:.2f} °C"
)
 print(f"Tendência: {tendencia}")
else:
 print(
"Não há leituras suficientes "
"para calcular a tendência."
)
```

Comparação entre leituras consecutivas

```
aumentos = 0
reducoes = 0
estabilidades = 0

for indice in range(
1,
 len(historico_temperatura)
):
 leitura_anterior = (
 historico_temperatura[indice - 1]
)

 leitura_atual = (
 historico_temperatura[indice]
)

 if leitura_atual > leitura_anterior:
 aumentos += 1
 elif leitura_atual < leitura_anterior:
 reducoes += 1
 else:
 estabilidades += 1
```

Exibição:

```
print()
print("COMPARAÇÃO ENTRE LEITURAS")
print("----------------------------------------")
print(f"Aumentos: {aumentos}")
print(f"Reduções: {reducoes}")
print(f"Estabilidades: {estabilidades}")
```

16. Programa completo de referência

Crie um arquivo chamado:

```
analise_historico.py
```

Utilize o programa abaixo como referência para integrar os desafios.

```

```

```
import random

print("========================================")
print(" SIMULADOR E ANALISADOR DE COLETA")
print("========================================")

codigo_equipamento = "M-01"
dispositivo_coleta = "CLP-SIM-01"
quantidade_ciclos = 20

limite_temperatura = 80
limite_pressao = 8
limite_corrente = 15

historico_ciclos = []
historico_temperatura = []
historico_pressao = []
historico_corrente = []
historico_estados = []

ciclos_normais = 0
ciclos_atencao = 0
ciclos_alarme = 0
ciclos_emergencia = 0

ciclos_realizados = 0
leituras_ignoradas = 0

print(f"Equipamento: {codigo_equipamento}")
print(f"Dispositivo: {dispositivo_coleta}")
print(f"Ciclos planejados: {quantidade_ciclos}")

for ciclo in range(quantidade_ciclos):
 numero_ciclo = ciclo + 1

 print()
 print(f"========== CICLO {numero_ciclo} ==========")

# Simulação de falha de comunicação
 if numero_ciclo == 5:
 print("Falha de comunicação com o sensor.")
 leituras_ignoradas += 1
 continue

# Simulação das leituras
 temperatura = random.randint(60, 95)
 pressao = random.uniform(5.0, 10.0)
 corrente = random.uniform(10.0, 20.0)
```

```
 ciclos_realizados += 1

# Classificação da temperatura
 if temperatura < 70:
 status_temperatura = "NORMAL"
 elif temperatura <= limite_temperatura:
 status_temperatura = "ATENÇÃO"
 else:
 status_temperatura = "ALARME"

# Classificação da pressão
 if pressao < 7:
 status_pressao = "NORMAL"
 elif pressao <= limite_pressao:
 status_pressao = "ATENÇÃO"
 else:
 status_pressao = "ALARME"

# Classificação da corrente
 if corrente < 13:
 status_corrente = "NORMAL"
 elif corrente <= limite_corrente:
 status_corrente = "ATENÇÃO"
 else:
 status_corrente = "ALARME"

# Situação crítica
 situacao_critica = (
 temperatura > 90
 or pressao > 10
 or corrente > 18
)

# Existência de alarme
 existe_alarme = (
 status_temperatura == "ALARME"
 or status_pressao == "ALARME"
 or status_corrente == "ALARME"
)

# Existência de atenção
 existe_atencao = (
 status_temperatura == "ATENÇÃO"
 or status_pressao == "ATENÇÃO"
 or status_corrente == "ATENÇÃO"
)
```

```
# Estado geral
 if situacao_critica:
 estado_geral = "EMERGÊNCIA"
 elif existe_alarme:
 estado_geral = "ALARME"
 elif existe_atencao:
 estado_geral = "ATENÇÃO"
 else:
 estado_geral = "NORMAL"

# Armazenamento da leitura válida
 historico_ciclos.append(numero_ciclo)
 historico_temperatura.append(temperatura)
 historico_pressao.append(pressao)
 historico_corrente.append(corrente)
 historico_estados.append(estado_geral)

# Contagem dos estados
 if estado_geral == "NORMAL":
 ciclos_normais += 1
 elif estado_geral == "ATENÇÃO":
 ciclos_atencao += 1
 elif estado_geral == "ALARME":
 ciclos_alarme += 1
 elif estado_geral == "EMERGÊNCIA":
 ciclos_emergencia += 1

# Exibição da leitura
 print(
 f"Temperatura: "
 f"{temperatura:.2f} °C - "
 f"{status_temperatura}"
)

 print(
 f"Pressão: "
 f"{pressao:.2f} bar - "
 f"{status_pressao}"
)

 print(
 f"Corrente: "
 f"{corrente:.2f} A - "
 f"{status_corrente}"
)

 print(f"Estado geral: {estado_geral}")
```

```
# Parada de emergência
 if estado_geral == "EMERGÊNCIA":
 print("PARADA DE EMERGÊNCIA ATIVADA.")
 break

print()
print("========================================")
print(" RESUMO DA COLETA")
print("========================================")
print(f"Equipamento: {codigo_equipamento}")
print(f"Dispositivo: {dispositivo_coleta}")
print(f"Ciclos planejados: {quantidade_ciclos}")
print(f"Ciclos realizados: {ciclos_realizados}")
print(f"Leituras ignoradas: {leituras_ignoradas}")
print(
 f"Leituras armazenadas: "
 f"{len(historico_ciclos)}"
)
print(f"Ciclos normais: {ciclos_normais}")
print(f"Ciclos em atenção: {ciclos_atencao}")
print(f"Ciclos em alarme: {ciclos_alarme}")
print(
 f"Ciclos em emergência: "
 f"{ciclos_emergencia}"
)

if len(historico_temperatura) > 0:
# Estatísticas da temperatura
 menor_temperatura = min(
 historico_temperatura
)

 maior_temperatura = max(
 historico_temperatura
)

 media_temperatura = (
 sum(historico_temperatura)
/ len(historico_temperatura)
)

# Estatísticas da pressão
 menor_pressao = min(historico_pressao)
 maior_pressao = max(historico_pressao)
 media_pressao = (
 sum(historico_pressao)
/ len(historico_pressao)
)
```

```
# Estatísticas da corrente
 menor_corrente = min(historico_corrente)
 maior_corrente = max(historico_corrente)
 media_corrente = (
 sum(historico_corrente)
/ len(historico_corrente)
)

 print()
 print("========================================")
 print(" ESTATÍSTICAS DO HISTÓRICO")
 print("========================================")
 print(
 f"Leituras armazenadas: "
 f"{len(historico_temperatura)}"
)

 print()
 print("TEMPERATURA")
 print(
 f"Mínima: "
 f"{menor_temperatura:.2f} °C"
)
 print(
 f"Máxima: "
 f"{maior_temperatura:.2f} °C"
)
 print(
 f"Média: "
 f"{media_temperatura:.2f} °C"
)

 print()
 print("PRESSÃO")
 print(
 f"Mínima: "
 f"{menor_pressao:.2f} bar"
)
 print(
 f"Máxima: "
 f"{maior_pressao:.2f} bar"
)
 print(
 f"Média: "
 f"{media_pressao:.2f} bar"
)
```

```
 print()
 print("CORRENTE")
 print(
 f"Mínima: "
 f"{menor_corrente:.2f} A"
)
 print(
 f"Máxima: "
 f"{maior_corrente:.2f} A"
)
 print(
 f"Média: "
 f"{media_corrente:.2f} A"
)

# Contagem de leituras acima dos limites
 leituras_temperatura_alta = 0
 leituras_pressao_alta = 0
 leituras_corrente_alta = 0

 for temperatura in historico_temperatura:
 if temperatura > limite_temperatura:
 leituras_temperatura_alta += 1

 for pressao in historico_pressao:
 if pressao > limite_pressao:
 leituras_pressao_alta += 1

 for corrente in historico_corrente:
 if corrente > limite_corrente:
 leituras_corrente_alta += 1

 print()
 print("LEITURAS ACIMA DOS LIMITES")
 print("----------------------------------------")
 print(
 f"Temperatura: "
 f"{leituras_temperatura_alta}"
)
 print(
 f"Pressão: "
 f"{leituras_pressao_alta}"
)
 print(
 f"Corrente: "
 f"{leituras_corrente_alta}"
)
```

```
# Percentual de temperaturas acima do limite
 total_leituras = len(
 historico_temperatura
)

 percentual_temperatura = (
 leituras_temperatura_alta
/ total_leituras
* 100
)

 print(
 f"Percentual de temperaturas acima "
 f"do limite: "
 f"{percentual_temperatura:.2f}%"
)

# Últimas leituras
 print()
 print("ÚLTIMAS LEITURAS")
 print("----------------------------------------")

 inicio_ultimas = max(
0,
 len(historico_ciclos) - 3
)

 for indice in range(
 inicio_ultimas,
 len(historico_ciclos)
):
 print(
 f"Ciclo: "
 f"{historico_ciclos[indice]}"
)

 print(
 f"Temperatura: "
 f"{historico_temperatura[indice]:.2f} °C"
)

 print(
 f"Pressão: "
 f"{historico_pressao[indice]:.2f} bar"
)

 print(
 f"Corrente: "
```

```
 f"{historico_corrente[indice]:.2f} A"
)

 print(
 f"Estado: "
 f"{historico_estados[indice]}"
)

 print()

# Maior temperatura e ciclo correspondente
 indice_maior_temperatura = (
 historico_temperatura.index(
 maior_temperatura
)
)

 ciclo_maior_temperatura = (
 historico_ciclos[
 indice_maior_temperatura
]
)

 print(
 f"Maior temperatura: "
 f"{maior_temperatura:.2f} °C"
)

 print(
 f"Ciclo da maior temperatura: "
 f"{ciclo_maior_temperatura}"
)

# Maior pressão e ciclo correspondente
 indice_maior_pressao = (
 historico_pressao.index(
 maior_pressao
)
)

 ciclo_maior_pressao = (
 historico_ciclos[
 indice_maior_pressao
]
)

 print(
 f"Maior pressão: "
```

```
 f"{maior_pressao:.2f} bar"
)

 print(
 f"Ciclo da maior pressão: "
 f"{ciclo_maior_pressao}"
)

# Maior corrente e ciclo correspondente
 indice_maior_corrente = (
 historico_corrente.index(
 maior_corrente
)
)

 ciclo_maior_corrente = (
 historico_ciclos[
 indice_maior_corrente
]
)

 print(
 f"Maior corrente: "
 f"{maior_corrente:.2f} A"
)

 print(
 f"Ciclo da maior corrente: "
 f"{ciclo_maior_corrente}"
)

# Tendência da temperatura
 if len(historico_temperatura) >= 2:
 primeira_temperatura = (
 historico_temperatura[0]
)

 ultima_temperatura = (
 historico_temperatura[-1]
)

 variacao_temperatura = (
 ultima_temperatura
- primeira_temperatura
)

 if variacao_temperatura > 0:
 tendencia = "AUMENTO"
```

```
 elif variacao_temperatura < 0:
 tendencia = "REDUÇÃO"
 else:
 tendencia = "ESTÁVEL"

 print()
 print("TENDÊNCIA DA TEMPERATURA")
 print("----------------------------------------")
 print(
 f"Primeira leitura: "
 f"{primeira_temperatura:.2f} °C"
)

 print(
 f"Última leitura: "
 f"{ultima_temperatura:.2f} °C"
)

 print(
 f"Variação: "
 f"{variacao_temperatura:.2f} °C"
)

 print(f"Tendência: {tendencia}")

# Comparação entre leituras consecutivas
 aumentos = 0
 reducoes = 0
 estabilidades = 0

 for indice in range(
1,
 len(historico_temperatura)
):
 leitura_anterior = (
 historico_temperatura[
 indice - 1
]
)

 leitura_atual = (
 historico_temperatura[indice]
)

 if leitura_atual > leitura_anterior:
 aumentos += 1
 elif leitura_atual < leitura_anterior:
 reducoes += 1
```

```
 else:
 estabilidades += 1

 print()
 print("COMPARAÇÃO ENTRE LEITURAS")
 print("----------------------------------------")
 print(f"Aumentos: {aumentos}")
 print(f"Reduções: {reducoes}")
 print(
 f"Estabilidades: "
 f"{estabilidades}"
)
else:
 print()
 print("Nenhuma leitura foi armazenada.")

print()
print("========================================")
print(" FIM DA ANÁLISE")
print("========================================")
```

17. Testes controlados

Como os valores simulados são aleatórios, os resultados podem variar a cada execução.

Para validar o funcionamento, substitua temporariamente a geração aleatória por valores conhecidos.

Teste 1 — Operação sem leituras acima dos limites

```
temperaturas_teste = [60, 65, 70, 75, 80]
pressoes_teste = [5.0, 5.5, 6.0, 6.5, 7.0]
correntes_teste = [10.0, 11.0, 12.0, 13.0, 14.0]
```

Dentro do laço:

```
temperatura = temperaturas_teste[ciclo]
pressao = pressoes_teste[ciclo]
corrente = correntes_teste[ciclo]
```

Resultados esperados para a temperatura:

```

```

```
Menor temperatura: 60.00 °C
Maior temperatura: 80.00 °C
Média da temperatura: 70.00 °C
```

Nenhuma temperatura deverá estar acima de 80 °C.

Teste 2 — Valores com alarme

```
temperaturas_teste = [65, 72, 84, 91, 78]
pressoes_teste = [5.0, 7.0, 8.5, 9.0, 6.5]
correntes_teste = [10.0, 13.0, 16.0, 19.0, 12.0]
```

Identifique:

• Quantas temperaturas estão acima de 80 °C;

• Quantas pressões estão acima de 8 bar;

• Quantas correntes estão acima de 15 A;

• Em qual ciclo ocorreu a maior temperatura;

• Em qual ciclo ocorreu a maior corrente;

• Se haverá uma situação de emergência.

Teste 3 — Poucas leituras

```
temperaturas_teste = [75]
pressoes_teste = [6.0]
correntes_teste = [12.0]
```

Verifique:

• O resultado das estatísticas;

• O comportamento do fatiamento `[-3:]`;

• O funcionamento da verificação de quantidade de leituras;

• A impossibilidade de comparar primeira e última leitura como tendência.

Teste 4 — Falha de comunicação

Configure o programa para ignorar o ciclo 3:

```
if numero_ciclo == 3:
 print("Falha de comunicação. Leitura ignorada.")
 leituras_ignoradas += 1
 continue
```

Verifique:

• Se o ciclo 3 ficou fora das listas;

• Se o número de leituras armazenadas foi reduzido;

• Se os números dos ciclos continuam corretos;

• Se a lista `historico_ciclos` preserva os ciclos válidos.

Teste 5 — Nenhuma leitura armazenada

Execute uma situação em que nenhuma leitura seja armazenada.

O programa deverá evitar erros e informar:

```
Nenhuma leitura foi armazenada.
```

18. Atividade de análise

Execute o programa pelo menos três vezes e registre os resultados.

Execução	Leituras armazenadas	Mínima	Máxima	Média	Acima do limite	Estado mais frequente
1						
2						
3						

Registre também:

Execução		Tendência	Leituras ignoradas	Ciclos realizados

	Ciclo da maior temperatura			
1				
2				
3				

19. Desafio de melhoria

Implemente uma das melhorias abaixo.

Opção A — Exibir somente os ciclos em alarme

```
for indice in range(
 len(historico_estados)
):
 estado = historico_estados[indice]

 if (
 estado == "ALARME"
 or estado == "EMERGÊNCIA"
):
 print()
 print(
 f"Ciclo: "
 f"{historico_ciclos[indice]}"
)
 print(
 f"Temperatura: "
 f"{historico_temperatura[indice]:.2f} °C"
)
 print(
 f"Pressão: "
 f"{historico_pressao[indice]:.2f} bar"
)
 print(
 f"Corrente: "
 f"{historico_corrente[indice]:.2f} A"
)
 print(f"Estado: {estado}")
```

Opção B — Encontrar a menor temperatura e seu ciclo

```
menor_temperatura = min(
 historico_temperatura
)

indice_menor_temperatura = (
 historico_temperatura.index(
 menor_temperatura
)
)

ciclo_menor_temperatura = (
 historico_ciclos[
 indice_menor_temperatura
]
)

print(
 f"Menor temperatura: "
 f"{menor_temperatura:.2f} °C"
)

print(
 f"Ciclo da menor temperatura: "
 f"{ciclo_menor_temperatura}"
)
```

Opção C — Calcular a amplitude térmica

A amplitude é a diferença entre o maior e o menor valor:

```
amplitude_temperatura = (
 maior_temperatura
- menor_temperatura
)

print(
 f"Amplitude térmica: "
 f"{amplitude_temperatura:.2f} °C"
)
```

Opção D — Criar uma lista de temperaturas críticas

```
temperaturas_criticas = []

for temperatura in historico_temperatura:
 if temperatura > limite_temperatura:
 temperaturas_criticas.append(
 temperatura
)

print("Temperaturas críticas:")

for temperatura in temperaturas_criticas:
 print(f"{temperatura:.2f} °C")
```

Opção E — Contar os estados armazenados

```
quantidade_normal = (
 historico_estados.count("NORMAL")
)

quantidade_atencao = (
 historico_estados.count("ATENÇÃO")
)

quantidade_alarme = (
 historico_estados.count("ALARME")
)

quantidade_emergencia = (
 historico_estados.count("EMERGÊNCIA")
)

print(f"Normais: {quantidade_normal}")
print(f"Atenção: {quantidade_atencao}")
print(f"Alarmes: {quantidade_alarme}")
print(
 f"Emergências: "
 f"{quantidade_emergencia}"
)
```

Opção F — Exibir o histórico em ordem inversa

```
for indice in range(
 len(historico_ciclos) - 1,
-1,
-1
):
 print(
 f"Ciclo: "
 f"{historico_ciclos[indice]}"
)
 print(
 f"Temperatura: "
 f"{historico_temperatura[indice]:.2f} °C"
)
 print(
 f"Estado: "
 f"{historico_estados[indice]}"
)
```

20. Entrega da atividade

Cada estudante ou dupla deverá entregar:

• Arquivo `analise_historico.py`;

• Histórico das leituras armazenado em listas;

• Lista com os números dos ciclos válidos;

• Análise estatística da temperatura;

• Análise estatística da pressão;

• Análise estatística da corrente;

• Contagem de leituras acima dos limites;

• Exibição das últimas leituras;

• Identificação do ciclo de maior temperatura;

• Identificação do ciclo de maior pressão;

• Identificação do ciclo de maior corrente;

• Análise simples da tendência da temperatura;

• Testes controlados;

• Uma melhoria adicional implementada.

21. Critérios de avaliação

Critério	Pontuação
Armazenamento correto das leituras	2,0
Uso adequado de listas e append()	1,5
Uso correto de índices e fatiamento	1,0
Exibição e percorrimento do histórico	1,0
Cálculo das estatísticas	2,0
Contagem de leituras críticas	1,0
Identificação dos ciclos críticos e análise de tendência	1,0
Testes, organização e clareza do código	0,5
Total	10,0

22. Fechamento da aula

Síntese dos conceitos

Na Aula 4, o programa passou a:

• Armazenar leituras em listas;

• Acessar elementos por índice;

• Utilizar índices negativos;

• Trabalhar com fatiamento;

• Percorrer históricos com `for`;

• Calcular mínimo, máximo e média;

• Contar ocorrências;

• Identificar ciclos críticos;

• Analisar a variação entre leituras;

• Produzir um relatório histórico.

Síntese da sequência didática

Na Aula 3, o sistema coletava os dados. Na Aula 4, ele passou a armazenar e analisar o histórico dessas coletas.

Na próxima etapa, os dados poderão ser organizados como registros completos, utilizando listas de dicionários para representar leituras, equipamentos e dispositivos de coleta.